

DATA-260 Homework 2 – Report

Uday Patel

Section 0 – Personal Configuration and Domain

Value	Result
SID4	9871
PORT_BASE	8871
PREFIX	s9871
SEED	9871
VERIFY_SEED	269871
DOMAIN_ID	7 – Community sports league fixtures

Hardware: Lenovo Legion Slim 5 16” (Ryzen 5 7640HS, 16GB RAM, NVIDIA RTX 4060 8GB, 512GB SSD)

Local model used: qwen3:8b, served via Ollama

Repository: <https://github.com/UdayPate/data260-9871>

Tagged commit: 2048435b2290d654906a7376c507f22e1f318206

<https://github.com/UdayPate/data260-9871/tree/2048435b2290d654906a7376c507f22e1f318206>

Part 1 – HTML & CSS: Responsive Layout and UI States

The HW1 fixture form and list were restyled with a vintage newspaper/box-score theme (`static/style.css`) and made usable at 375px width. A Fixture Board list section with visible loading, empty, and error states was added.

375px responsiveness

The key mechanism is a media query that reflows the fixture table into stacked cards on narrow screens, using each cell's `data-label` attribute to show which field is which without column headers:

```
@media (max-width: 480px) {
  body { padding: 8px; }
  .masthead h1 { font-size: 1.3rem; letter-spacing: 0.03em; }
  form { padding: 14px; }

  /* Reflow the box-score table into stacked cards on narrow screens */
  table.fixture-table thead { display: none; }

  table.fixture-table,
  table.fixture-table tbody,
  table.fixture-table tr,
  table.fixture-table td {
    display: block;
    width: 100%;
  }

  table.fixture-table tr {
    border: 1px solid var(--rule);
    margin-bottom: 10px;
    padding: 8px;
  }

  table.fixture-table td { border: none; padding: 4px 0; }

  table.fixture-table td::before {
    content: attr(data-label) ": ";
    font-weight: bold;
    text-transform: uppercase;
    font-size: 0.7rem;
    color: var(--ink-light);
  }
}
```

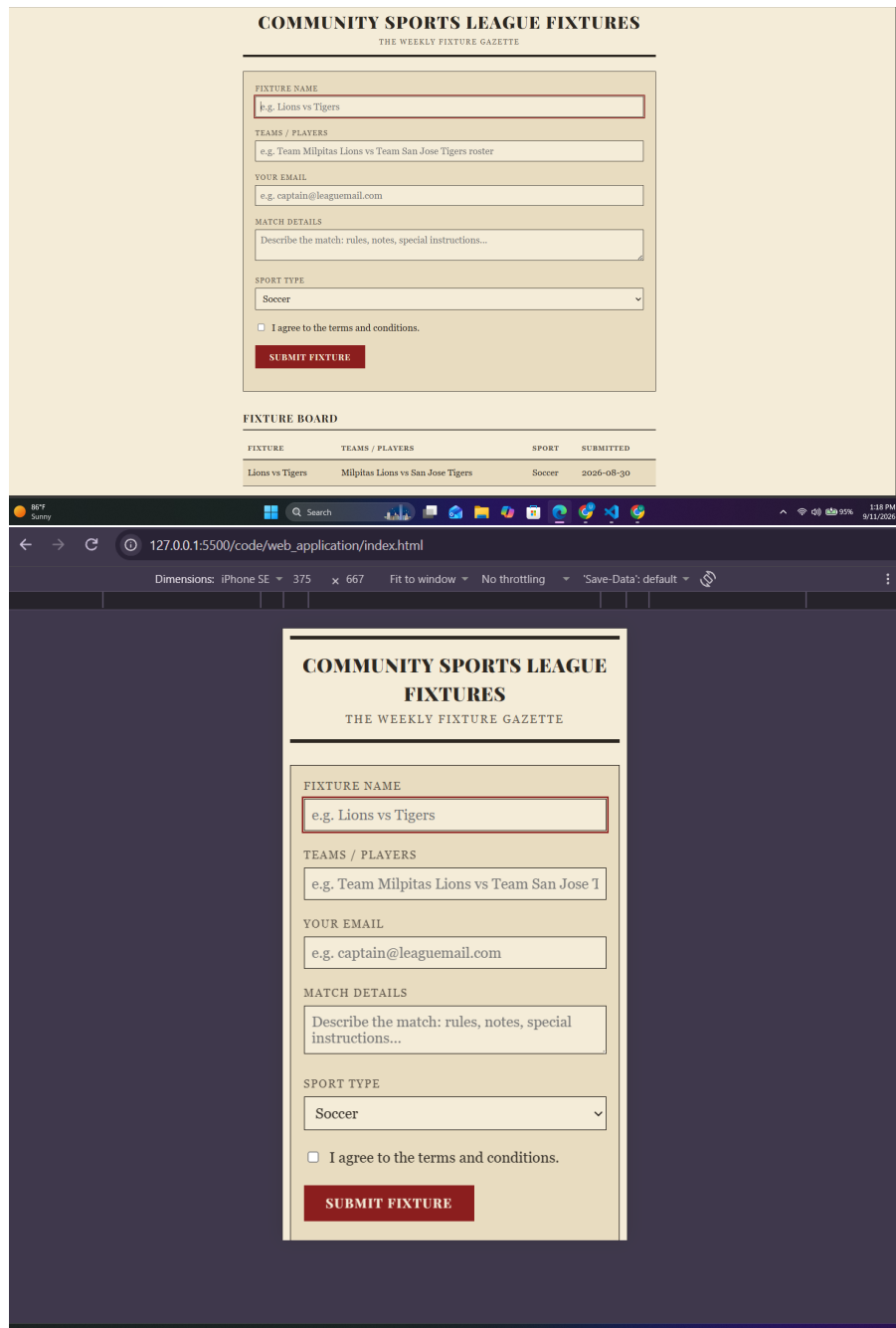


Figure 1: The application at desktop width (top) and at exactly 375px via Chrome DevTools' iPhone SE preset (Bottom). The table reflows into labeled stacked cards rather than squeezing columns.

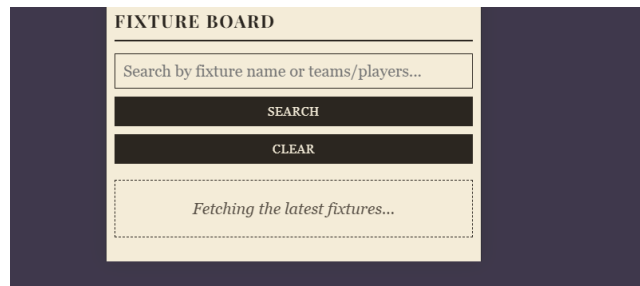
Loading, empty, and error states

Three state boxes are hidden by default and toggled visible by JavaScript depending on the outcome of the fetch to the FastAPI JSON API:

```
<div id="loadingState" class="state-box state-loading">
  Fetching the latest fixtures&hellip;
</div>
<div id="emptyState" class="state-box state-empty">
  No fixtures on the docket yet. Submit one above to get started.
</div>
<div id="errorState" class="state-box state-error">
  Wire report failed &mdash; unable to load fixtures. Please try again.
</div>
```

```
function setState(state) {
  [loadingState, emptyState, errorState].forEach((el) => el.classList.remove("active"));
  fixtureTable.style.display = "none";
  if (state === "loading") loadingState.classList.add("active");
  if (state === "empty") emptyState.classList.add("active");
  if (state === "error") errorState.classList.add("active");
  if (state === "loaded") fixtureTable.style.display = "";
}

async function loadFixtures(query) {
  setState("loading");
  try {
    const url = query ? `/api/fixtures?q=${encodeURIComponent(query)}` : "/api/fixtures";
    const response = await fetch(url);
    if (!response.ok) throw new Error(`Request failed with status ${response.status}`);
    const fixtures = await response.json();
    if (fixtures.length === 0) { setState("empty"); }
    else { renderFixtures(fixtures); setState("loaded"); }
  } catch (err) {
    console.error("Failed to load fixtures:", err);
    setState("error");
  }
}
```



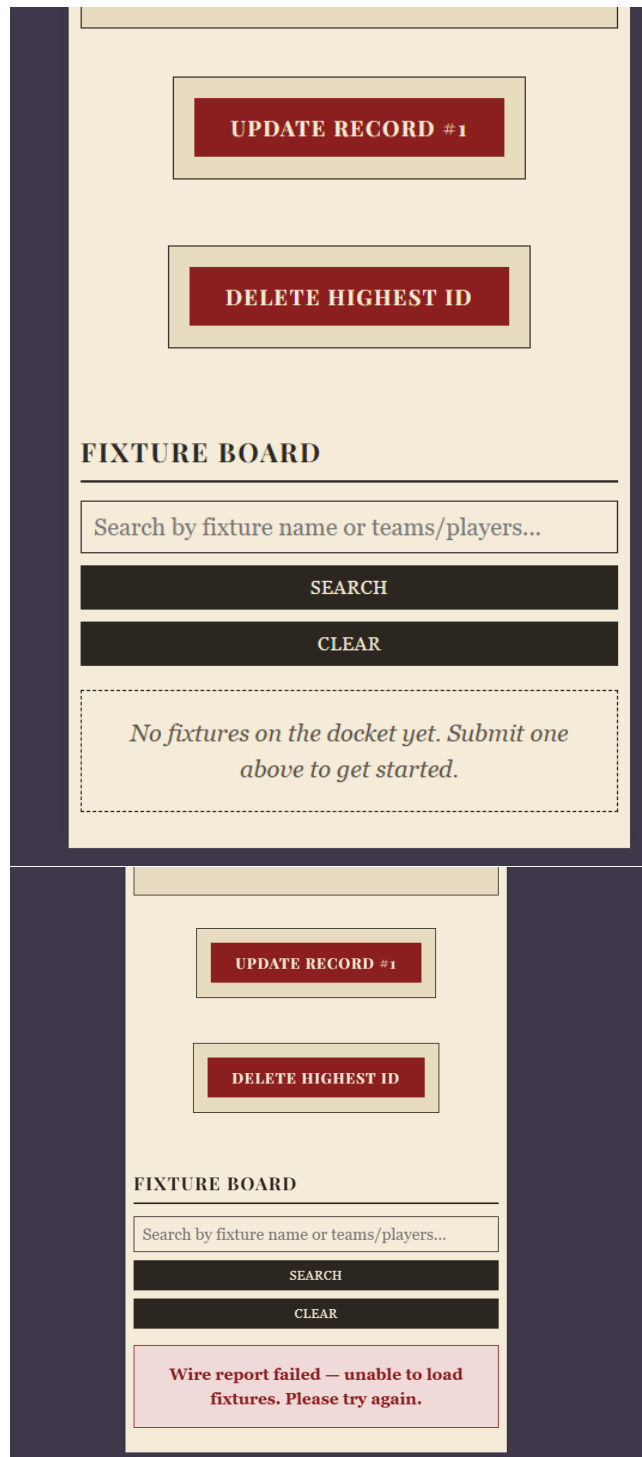


Figure 2: Loading state (network throttled via DevTools, Top), empty state (after deleting all records, center), and error state (`/api/fixtures` blocked via DevTools, Bottom).

Part 2 – FastAPI Backend

A FastAPI backend (`code/web_application/main.py`) runs on `PORT_BASE` (8871), using an in-memory list for storage and Jinja2 templates for the home view. All four required operations use classic POST + redirect, with the fixture list itself loaded client-side via a small JSON API.

1. Add a new record

```
@app.post("/fixtures/create")
def create_fixture(
    fixture_name: str = Form(...),
    teams_players: str = Form(...),
    submitter_email: str = Form(...),
    description: str = Form(...),
    category: str = Form(...),
    agree_terms: Optional[str] = Form(None),
):
    global next_id
    fixtures.append({
        "id": next_id,
        "fixture_name": fixture_name,
        "teams_players": teams_players,
        "submitter_email": submitter_email,
        "description": description,
        "category": category,
        "agree_terms": agree_terms == "on",
        "submitted": now_iso(),
    })
    next_id += 1
    return RedirectResponse(url="/", status_code=303)
```

COMMUNITY SPORTS LEAGUE FIXTURES
THE WEEKLY FIXTURE GAZETTE

FIXTURE NAME
e.g. Lions vs Tigers

TEAMS / PLAYERS
e.g. Team Milpitas Lions vs Team San Jose Tigers roster

YOUR EMAIL
e.g. captain@leagueemail.com

MATCH DETAILS
Describe the match: rules, notes, special instructions...

SPORT TYPE
Soccer

☐ I agree to the terms and conditions.

SUBMIT FIXTURE

UPDATE RECORD <-1 **DELETE HIGHEST ID**

FIXTURE BOARD

Search by fixture name or teams/players... **SEARCH** **CLEAR**

ID	FIXTURE	TEAMS / PLAYERS	SPORT	SUBMITTED
1	Lions vs Tigers	Milpitas Lions vs San Jose Tigers	soccer	2026-09-11 21:11 UTC
2	Warriors vs Strikers	Milpitas Warriors vs San Jose Strikers	cricket	2026-09-11 21:11 UTC

Figure 3: Fixture Board after submitting the create form – a new record (ID 3, "SJ vs SF") appears, confirming the redirect-and-refetch flow.

2. Update the record with ID 1

```

@app.post("/fixtures/update-first")
def update_first():
    for f in fixtures:
        if f["id"] == 1:
            f["fixture_name"] = "Lions vs Tigers - RESCHEDULED"
            f["teams_players"] = "Milpitas Lions vs San Jose Tigers (new roster)"
            f["submitter_email"] = "league-office@leaguemail.com"
            f["description"] = "Match moved to Sunday due to field maintenance."
            f["category"] = "soccer"
            f["agree_terms"] = True
            f["submitted"] = now_iso()
            break
    return RedirectResponse(url="/", status_code=303)

```

The screenshot shows a web application interface for managing fixtures. At the top, there is a form with the following fields:

- FIXTURE NAME:** A text input field containing "e.g. Lions vs Tigers".
- TEAMS / PLAYERS:** A text input field containing "e.g. Team Milpitas Lions vs Team San Jose Tigers roster".
- YOUR EMAIL:** A text input field containing "e.g. captain@leaguemail.com".
- MATCH DETAILS:** A text area containing "Describe the match: rules, notes, special instructions...".
- SPORT TYPE:** A dropdown menu with "Soccer" selected.
- Agree to the terms and conditions:** A checkbox that is checked.
- SUBMIT FIXTURE:** A red button.

Below the form, there are two buttons: "UPDATE RECORD #1" and "DELETE HIGHEST ID".

At the bottom, there is a section titled "FIXTURE BOARD" with a search bar and a table of fixtures.

ID	FIXTURE	TEAMS / PLAYERS	SPORT	SUBMITTED
1	Lions vs Tigers - RESCHEDULED	Milpitas Lions vs San Jose Tigers (new roster)	soccer	2025-09-11 21:13 UTC
2	Warriors vs Strikers	Milpitas Warriors vs San Jose Strikers	cricket	2025-09-11 21:11 UTC
3	SJ vs SF	San Jose Falcons VS Bay City Titans	basketball	2025-09-11 21:13 UTC

Figure 4: Record ID 1 renamed to "Lions vs Tigers - RESCHEDULED" with all fields updated, confirming the fixed one-click update action.

3. Delete the record with the highest ID

```
@app.post("/fixtures/delete-highest")
def delete_highest():
    if fixtures:
        highest = max(fixtures, key=lambda f: f["id"])
        fixtures.remove(highest)
    return RedirectResponse(url="/", status_code=303)
```

COMMUNITY SPORTS LEAGUE FIXTURES
THE WEEKLY FIXTURE GAZETTE

FIXTURE NAME
e.g. Lions vs Tigers

TEAMS / PLAYERS
e.g. Team Milpitas Lions vs Team San Jose Tigers roster

YOUR EMAIL
e.g. captain@leagueemail.com

MATCH DETAILS
Describe the match: rules, notes, special instructions...

SPORT TYPE
Soccer

☐ I agree to the terms and conditions.

SUBMIT FIXTURE

UPDATE RECORD +1

DELETE HIGHEST ID

FIXTURE BOARD

Search by fixture name or teams/players... SEARCH CLEAR

ID	FIXTURE	TEAMS / PLAYERS	SPORT	SUBMITTED
1	Lions vs Tigers	Milpitas Lions vs San Jose Tigers	soccer	2026-09-12 01:23 UTC

Figure 5: After clicking "Delete Highest ID," record ID 2 has been removed, leaving only record ID 1.

4. Search by primary or secondary field

```
@app.get("/api/fixtures")
def list_fixtures(q: Optional[str] = None):
    if q:
        q_lower = q.lower()
        results = [
            f for f in fixtures
            if q_lower in f["fixture_name"].lower()
            or q_lower in f["teams_players"].lower()
        ]
    else:
        results = fixtures
    return JSONResponse(content=results)
```

```
searchButton.addEventListener("click", () => {
    loadFixtures(searchInput.value.trim());
});
```

UPDATE RECORD +1

DELETE HIGHEST ID

FIXTURE BOARD

Warriors SEARCH CLEAR

ID	FIXTURE	TEAMS / PLAYERS	SPORT	SUBMITTED
2	Warriors vs Strikers	Milpitas Warriors vs San Jose Strikers	cricket	2026-09-11 21:11 UTC

Figure 6: Searching "Warriors" correctly filters the list to the one matching record.

Part 3 – Stateful Agent Graph (LangGraph)

The HW1 sequential Planner → Reviewer script was refactored into a stateful LangGraph (code/agent_graph.py) implementing the supervisor pattern. All LLM calls route through src/model_client.py's ModelClient adapter, per the assignment's requirement.

Shared state

```
class AgentState(TypedDict):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any
    planner_proposal: Dict[str, Any]
    reviewer_feedback: Dict[str, Any]
    turn_count: int
    turn_ceiling: int
    schema_error: str
```

Planner node (calls the adapter, clears stale feedback)

```
def planner_node(state: AgentState) -> Dict[str, Any]:
    llm: ModelClient = state["llm"]
    # ... builds prompt, incorporating schema_error or reviewer issues if present ...
    raw = llm.complete(messages)
    proposal = extract_json(raw)
    return {"planner_proposal": proposal, "reviewer_feedback": {}}
```

Reviewer node

```
def reviewer_node(state: AgentState) -> Dict[str, Any]:
    llm: ModelClient = state["llm"]
    proposal = state["planner_proposal"]
    # ... builds a review prompt checking tag count/specificity, summary length ...
    raw = llm.complete(messages)
    feedback = extract_json(raw)
    return {"reviewer_feedback": feedback}
```

Supervisor node (only job: update the turn counter)

```
def supervisor_node(state: AgentState) -> Dict[str, Any]:
    new_turn_count = state.get("turn_count", 0) + 1
    return {"turn_count": new_turn_count}
```

Router logic (reads state, decides the next node)

```
def router_logic(state: AgentState) -> str:
    ceiling = state.get("turn_ceiling", 6)
    if state["turn_count"] >= ceiling:
        return "end"
    if not state.get("planner_proposal"):
        return "planner"
    feedback = state.get("reviewer_feedback")
```

```
if not feedback:
    return "reviewer"
if feedback.get("issues"):
    return "planner" # self-correction loop
return "end"
```

Verified run: organic self-correction loop

This is a real run against qwen3:8b (not a simulated/forced test) where the Reviewer genuinely caught a generic tag and the Planner genuinely fixed it:

```
---NODE: Supervisor (turn 1)---
[ROUTER] no proposal yet -> planner
---NODE: Planner---
Proposal: {'tags': ['bay area rivalry', 'cricket ground south', 'sports event'],
'summary': 'San Jose Strikers and Milpitas Warriors renew their Bay Area
rivalry at Cricket Ground South this weekend.'}
---NODE: Supervisor (turn 2)---
[ROUTER] has proposal, not yet reviewed -> reviewer
---NODE: Reviewer---
Feedback: {'issues': ["tags include generic term 'sports event'"]}
---NODE: Supervisor (turn 3)---
[ROUTER] reviewer found issues [...] -> planner (retry)
---NODE: Planner---
Proposal: {'tags': ['bay area rivalry', 'soccer game', 'local teams'],
'summary': 'San Jose Strikers and Milpitas Warriors renew their Bay Area
soccer rivalry this weekend.'}
---NODE: Supervisor (turn 4)---
[ROUTER] has proposal, not yet reviewed -> reviewer
---NODE: Reviewer---
Feedback: {'issues': []}
---NODE: Supervisor (turn 5)---
[ROUTER] reviewed, no issues -> END
```

Verified run: forced-issue ceiling test

`reviewer_node` was temporarily hardcoded to always report an issue (skipping the LLM call), to confirm the `turn_ceiling` safety valve actually stops the loop rather than running forever:

```
---NODE: Supervisor (turn 1)--- -> planner
---NODE: Planner--- (tags: bay area rivalry, cricket ground south, team match)
---NODE: Supervisor (turn 2)--- -> reviewer
---NODE: Reviewer--- [FORCED TEST MODE] always reporting an issue
---NODE: Supervisor (turn 3)--- reviewer found issues -> planner (retry)
---NODE: Planner--- (tags: bay area rivalry, soccer match, cricket ground)
---NODE: Supervisor (turn 4)--- -> reviewer
---NODE: Reviewer--- [FORCED TEST MODE] always reporting an issue
---NODE: Supervisor (turn 5)--- reviewer found issues -> planner (retry)
---NODE: Planner--- (tags: bay area rivalry, san jose strikers, milpitas warriors)
---NODE: Supervisor (turn 6)---
[ROUTER] turn_count=6 >= ceiling=6 -> END
```

The graph correctly stopped at `turn_count=6` (matching `turn_ceiling=6`) rather than looping forever, even though the Reviewer would have kept reporting issues indefinitely.

Part 4 – Output Schema and Loop Safety

Pydantic validation

Exactly 3 string tags (3-30 characters each) and a summary of at most 25 words, enforced before a proposal is ever sent to the Reviewer:

```
class TagsSummarySchema(BaseModel):
    tags: List[str] = Field(..., min_length=3, max_length=3)
    summary: str

    @field_validator("tags")
    @classmethod
    def check_tag_lengths(cls, tags: List[str]) -> List[str]:
        for tag in tags:
            if not (3 <= len(tag) <= 30):
                raise ValueError(f"tag {tag!r} is {len(tag)} characters; "
                                f"each tag must be 3-30 characters")
        return tags

    @field_validator("summary")
    @classmethod
    def check_summary_word_count(cls, summary: str) -> str:
        word_count = len(summary.split())
        if word_count > 25:
            raise ValueError(f"summary has {word_count} words; must be at most 25 words")
        return summary
```

On failure, the specific Pydantic error message is fed directly back into the Planner’s next prompt for a retry – verified with a real failure during Experiment 2 (**ceiling=10**, run 2): a 26-word summary was rejected, fed back, and corrected on the very next attempt.

Experiment 1: 30-run classification (turn_ceiling=6)

Outcome	Count	Mean latency (ms)
Valid first attempt	25	35,777.8
Valid after 1 retry	4	80,595.1
Valid after 2+ retries	0	–
Hit turn ceiling	1	81,672.1

Experiment 2: Turn ceiling comparison (20 runs each)

Metric	Ceiling = 2	Ceiling = 10
Completion rate	0% (0/20)	100% (20/20)
Mean latency (ms)	17,526.5	46,256.8

Why ceiling=2 always fails: the Supervisor node increments **turn_count** on every visit, including its very first entry and its check-in immediately after Planner’s first attempt. By the time Supervisor re-evaluates after Planner’s first proposal, **turn_count** already equals 2, forcing END before the Reviewer node ever runs once – this is a structural limitation, not a model-quality issue.

Deployment choice: **turn_ceiling=10** (or any value ≥ 6 , per Experiment 1) should be used for deployment. Ceiling=2 guarantees 0% completion regardless of model quality.

Experiment 3: Adversarial input (5 runs, turn_ceiling=6)

Adversarial input: title “Match”, content “A game happened. It was fine.” – chosen for its near-total lack of specific information to derive tags from.

Outcome	Count
Hit turn ceiling	2 (40%)
Success	3 (60%)

This did not reach the ceiling in 4+ of 5 runs, so the observed rate (40%) is reported rather than a claim of deterministic failure. It was nonetheless clearly harder than the normal input: the Reviewer intervened in 4 of 5 runs here (80%) vs. only 5 of 30 runs (16.7%) in Experiment 1.

Why this input causes trouble: the Reviewer’s instructions ask for tags “genuinely specific to the content,” but this input contains no specific entities, teams, locations, or details to draw from – the Planner is caught between honestly generic tags (rejected by the Reviewer) and inventing ungrounded specific-sounding details.

Proposed fix: add an input-validation step before the graph starts – e.g., a minimum content length or a check for at least one proper noun/specific entity – and short-circuit with a clear “insufficient detail” message, rather than letting Planner/Reviewer loop repeatedly attempting to extract specificity that was never there.